

Akash Ravi Sontakke

Power Platform Developer & Automation Engineer (Python / PHP / MySQL / HTML)

Case studies: Power Apps, Dataverse, Power Automate, Power BI, SharePoint, Azure AD (MSAL) & Microsoft Graph projects, plus Python/pandas automation, and PHP, MySQL & HTML/JavaScript web application development

This document summarizes eleven internal enterprise applications, reports, and tools I designed and built end-to-end — six Power Platform apps, three Power BI reporting dashboards, one Python automation tool, and one full-stack PHP/MySQL web application. Each write-up covers the business problem, my role, the technical architecture, and the measured impact — without exposing the employer's name, live URLs, real employee data, or any information that could be commercially sensitive.

Live application links, real screenshots with employee data, and source code are intentionally withheld or redacted here; they can be shared selectively (e.g. a live walkthrough) during an interview upon request. The Python automation project's source code is public on GitHub and linked in its case study.

Contents

POWER PLATFORM / POWER APPS PROJECTS	
1.	Establishment/HR Dept. Employee Data Management System
2.	Digital Signature System
3.	Secure Internal Communication Platform
4.	Organization Project Manager Portal
5.	FD Commission & Payout Management App
6.	Automated Leave Management System
POWER BI REPORTING PROJECTS	
7.	Credit Control & Recovery Dashboard
8.	Insurance Policy Management Dashboard
9.	Head Office Daily Productivity Report
PYTHON / DATA AUTOMATION PROJECTS	
10.	Overdue Accounts Report Generator
WEB DEVELOPMENT / PHP & MYSQL PROJECTS	
11.	Weekly Meeting Tracker
APPENDIX	
—.	A Note on Confidentiality

1. Establishment/HR Dept. Employee Data Management System

Centralized Power Apps portal for HR & Establishment teams to manage the complete employee lifecycle — onboarding, service book, increments, loans, deputations & every type of HR letter

Problem

This Power Apps portal is built for the HR and Establishment departments to maintain complete employee data in one place and generate every type of HR letter from it. Before this system, employee records for a multi-branch organization were maintained manually across spreadsheets, with no centralized system covering service history, increments, loans, deputations, or HR letter generation — record-keeping was slow and error-prone, and department leadership had no unified, real-time view of the workforce or document compliance.

My Role

Sole developer, end-to-end: designed the data model across two SharePoint site collections, built the Canvas App covering the full employee lifecycle for both HR and Establishment functions, and configured every Power Automate approval and notification flow.

Architecture

Layer	Technology
Application	Microsoft Power Apps (Canvas App)
Backend / Data	SharePoint Online (multiple lists, 2 site collections)
Automation	Power Automate (Logic Flows)
Approvals	Power Automate Approvals connector
Notifications	Office 365 Outlook connector
File Storage	OneDrive for Business (deputation files, generated PDFs)

Key Features

- Secure employee login validated against a SharePoint credentials list
- Employee master data: add, edit, and browse full records across all branches
- Combined Profile & Service Book view for each employee
- Paste-CSV bulk data import for fast, large-batch employee record uploads
- Increment tracking with bulk-dispatch of increment letters by email
- HR Letter Generator covering every letter type — warning letter, show-cause notice, transfer order, experience & relieving, resignation acceptance, termination letter, and more — with a live A4 preview, built-in approval workflow, and auto-saved PDFs
- End-to-end deputation letter and order management between branches
- Employee loan tracking, plus one-click export of data to Excel

App Screens

- Staff Login & Workforce Dashboard (headcount, joiners/leavers, document compliance tracker)
- Employee Data list/browse, Add New Employee, Employee Edit Page

- Employee Digital Profile (combined profile, service book & deputation history) with a per-employee Document Checklist
- Deputation Order creation and routing screen
- HR Letter Dashboard, HR Letter Generator, and Bulk Increment Email Dispatcher
- Paste-CSV Sync Tool for bulk employee uploads

Automated Flows

- Export Employee Data to Excel — one-click export of employee records
- Increment Bulk Sender — bulk email dispatch for increment letters
- HR Letter Approval — routes generated letters through an approval flow before finalizing
- Save HR Letter PDF — auto-generates and saves letter PDFs on approval
- Deputation File — automates deputation document creation and storage

Impact

- Gave HR and Establishment a single system to maintain all employee data and issue every type of HR letter, replacing manual, spreadsheet-based record-keeping across every branch
- Gave Establishment/HR Dept leadership a real-time workforce dashboard for headcount and document compliance, instead of static registers
- Cut turnaround time on HR letters and deputation orders with built-in approval routing and auto-generated PDFs

2. Digital Signature System

Paperless document signing & branch authorization platform

Problem

Branch staff at a multi-state credit co-operative society (100+ branches) relied on printing, physically signing, and couriering authorization documents to Head Office — slow, hard to track, and with no central visibility into which branches or devices were authorized to transact. IT/Establishment had no single system to manage branch authorizations or verify that a signed document actually came from an approved device.

My Role

Sole developer. Designed and built both halves of the system from scratch: the Branch Portal (used by branch staff) and the Admin Panel (used by IT/Establishment).

Architecture

Layer	Technology
Frontend	HTML, CSS, JavaScript (no framework, no build step)
Authentication	Microsoft MSAL (Azure AD)
Database	SharePoint Lists & Libraries via Microsoft Graph API
PDF Handling	Client-side PDF render, sign, and download
Hosting	Static site (GitHub Pages / any HTTPS host)

Key Features

- Login via Microsoft 365 SSO or Employee ID + Password, with first-time/permanent password setup
- Device / IP authorization check before granting branch-level access
- View, preview, and digitally sign pending documents (draw or upload signature)
- Personal signing history log for every user
- Idle-timeout auto logout for security
- Admin: branch management (add/edit/close, transfer branch managers), bulk CSV branch upload with progress tracking
- Admin: full branch “document journey” timeline, exportable to CSV/PDF
- Admin: document verification queue and branch-wise reporting (Excel/PDF export)

Note — why static hosting: This app is deployed as a static site (GitHub Pages / any HTTPS host) rather than to a managed Azure service, because the organization did not provision paid Azure hosting. With no backend budget available, the entire app was architected as client-side only, using SharePoint + Microsoft Graph as the data layer so a free static host is sufficient.

Impact

- Eliminated a fully paper-based signature/authorization process across 100+ branches
- Gave IT/Establishment centralized, auditable control over which branches and devices can sign documents
- Every signature now has a timestamped, traceable history — previously unavailable with the paper process

3. Secure Internal Communication Platform

Auditable branch-to-branch messaging, replacing WhatsApp

Problem

Branch-to-branch coordination was happening over WhatsApp — a consumer app with no audit trail, no access control tied to employment status, and organizational data (project updates, files, financial figures) leaving the company's security boundary entirely.

My Role

Sole developer, end-to-end: UI, authentication integration, and the SharePoint-backed data layer.

Architecture

Layer	Technology
Frontend	HTML, CSS, JavaScript
Authentication	Microsoft MSAL (Azure AD) — organizational account required
Database	SharePoint Lists via Microsoft Graph API
Hosting	GitHub Pages

Key Features

- Branch-to-branch messaging and file sharing inside a controlled, single-sign-on environment
- Dashboard with chat overview and unread/activity summary
- Approval workflow module for routing requests through the platform
- Position/status tracking module for operational figures relevant to daily branch reporting
- User profile management

Note — why static hosting: Like the Digital Signature System, this app is hosted as a static site because the organization did not provide a paid Azure hosting service. The app is fully client-side, so a static host is all that's needed.

Impact

- Moved sensitive internal coordination out of a consumer chat app and inside the Microsoft 365 security boundary
- Introduced an auditable record of branch communications that didn't exist before
- Access is automatically revoked when an employee's organizational account is deactivated — unlike WhatsApp

4. Organization Project Manager Portal

Central tracker for cross-team Power Platform initiatives

Problem

With multiple Power Apps builds, dashboards, and cross-team initiatives running in parallel, there was no single place for leadership to see project status, deadlines, or ownership — status updates were being gathered manually team by team.

My Role

Sole developer, end-to-end: information architecture, role-based views, and the reporting/export layer.

Architecture

Layer	Technology
Frontend	HTML, CSS, JavaScript
Authentication	Microsoft MSAL (Azure AD)
Database	SharePoint Lists via Microsoft Graph API
Hosting	GitHub Pages

Key Features

- Role-based access: Admin, Team Member, and Higher Authority (view-only)
- Live project tracking with progress %, deadlines, and overdue alerts
- Gantt-style timeline dashboard for at-a-glance scheduling
- Master report view with CSV export for offline analysis
- Dark mode support

Note — why static hosting: As with the two apps above, this portal is hosted as a static site since the organization did not provide a paid Azure hosting service. SharePoint + Graph handles all data, so no server-side hosting is required.

Impact

- Gave leadership self-serve visibility into project status instead of manual reporting cycles
- Overdue-alert logic surfaced at-risk projects earlier than the previous manual process
- Centralized reporting replaced scattered spreadsheets across teams

5. FD Commission & Payout Management App

Branch-level fixed-deposit commission tracking with approval workflow

Problem

Fixed-deposit (FD) commission and payout requests submitted by branches were tracked manually, with no consistent calculation method, no approval trail for finance, and no daily consolidated view of submissions across branches.

My Role

Sole developer. Built the Canvas App, the commission-calculation logic, and the Power Automate approval flow.

Architecture

Layer	Technology
Frontend	Power Apps (Canvas App)
Backend / Data	SharePoint Online
Automation	Power Automate approval workflow
Logic	Power Fx

Key Features

- Branch-based login with role-based visibility of requests
- FD payout request entry with built-in commission calculation
- Daily submission summary view for supervisors
- Automated Power Automate approval routing

Impact

- Standardized commission calculation across all branches, removing manual/inconsistent computation
- Introduced a traceable, auditable approval trail for every payout request
- Gave supervisors a daily consolidated view instead of collecting figures branch by branch

6. Automated Leave Management System

End-to-end digital leave request & approval portal

Problem

Leave requests across 100+ branches were tracked manually with no automated approval routing, no real-time status visibility for employees, and frequent data-entry errors from re-typing employee details by hand.

My Role

Power Platform Developer. Designed the data model, built the Canvas App UI, and authored the Power Automate approval engine.

Architecture

Layer	Technology
Frontend	Power Apps (Canvas App)
Backend / Data	Microsoft Dataverse — relational tables with filtering logic for real-time, user-specific views
Automation	Power Automate cloud flow for multi-stage Approve/Reject routing and email notifications
Identity	Microsoft 365 (User()) function for auto-login and profile photo integration)

Key Features

- Automatic login detection and personalized access using the signed-in Microsoft 365 identity
- Multi-stage manager approval workflow with automated email notifications on Approve/Reject
- Real-time “Ticket Journey” dashboard so employees can track their own request status without contacting HR
- Auto-populated employee details pulled directly from the organization's directory, reducing manual entry
- High-contrast, modern UI (glassmorphism/neumorphism styling) designed for non-technical branch staff
- Collections + Refresh()-based data loading tuned for performance under concurrent branch usage

Impact

- Removed paper-based leave requests across the entire multi-branch organization
- Gave employees self-serve visibility into approval status, reducing HR's manual status-update workload
- Reduced data-entry errors by auto-populating employee details instead of manual input

7. Credit Control & Recovery Dashboard

Power BI reporting suite for loan recovery, collections & field-visit performance

Problem

The Credit Control department at a multi-state credit co-operative society (100+ branches) needed a consolidated way to track loan recovery performance across all branches and loan types (vehicle, housing, personal, cash credit, micro-finance, etc.), alongside the productivity of the telecalling and field-visit (CFO) recovery teams. This data previously lived across separate exports with no unified, drillable view for management.

My Role

Sole developer. Modeled the data, built all DAX measures, and designed the full multi-page report suite end-to-end.

Architecture

Layer	Technology
Tool	Power BI Desktop
Data Modeling	Power Query (M) for transformation, star-schema data model
Calculations	DAX measures for recovery totals, OTS (One-Time Settlement) tracking, call/visit productivity
Sources	Internal loan/collections data exports, consolidated and refreshed into the model

Key Features

- Executive dashboard: total recovery, customer counts, OTS/regular/partial/closed account status, latest call & visit dates
- Branch and loan-type productivity views broken down by product (vehicle, housing, personal, cash credit, warehouse, micro-finance loans)
- Telecalling analytics: answered / unanswered / disconnected call volumes, daily and monthly trends
- Field-visit (CFO) tracking: visit counts, task completion status, visits created by telecallers vs. field staff
- Daily and year-wise branch recovery reports with account-closure and recovery-amount trends
- Loan GL balance summary by account type and branch
- Interactive slicers for year, branch, and loan-type filtering across every page

Impact

- Gave management a single, drillable view of recovery performance instead of manually consolidating branch-level exports
- Made telecalling and field-visit productivity visible and comparable across branches for the first time
- Daily recovery reporting replaced a previously manual, spreadsheet-based process

8. Insurance Policy Management Dashboard

Power BI reporting for customer loan insurance, vehicle & contingency policies

Problem

Insurance policies tied to customer loans, company vehicles, employee/director coverage, and contingency policies were tracked with no central reporting layer — making it hard to see policy renewal dates, coverage counts by branch, or exposure by insurance company and policy type at a glance.

My Role

Sole developer. Built the data model and full multi-page report from raw policy records.

Architecture

Layer	Technology
Tool	Power BI Desktop
Data Modeling	Power Query (M) for transformation, DAX for aggregated policy metrics
Sources	Internal policy/insurance records, consolidated and refreshed into the model

Key Features

- Customer loan insurance: branch-wise policy counts, GL-wise policy breakdown, top-10 branch ranking, full policy detail table
- Director & employee coverage dashboard: policy counts by insurance company and policy type
- Vehicle insurance tracking with renewal trend visuals
- Contingency policy tracking: firm policy renewal dates, sum-insured breakdown by policy sub-type
- Company-wide policy-type distribution and searchable policy-number lookup

Impact

- Replaced scattered policy records with a single reportable view across all policy categories
- Made upcoming renewal dates visible in advance, reducing the risk of lapsed coverage
- Gave leadership a branch-level and company-level view of insurance exposure that didn't exist before

9. Head Office Daily Productivity Report

Power BI dashboard for department-wise, employee-level daily output tracking

Problem

Head Office departments (Credit, HMPL, Accounts Payable & Insurance, and the Recovery/Reconciliation desk) had no consistent way to see individual employee output on a given day — work was logged informally, with no manager-level visibility into volumes handled per employee or per department.

My Role

Sole developer. Built the data model and single-page productivity report from daily work-log entries.

Architecture

Layer	Technology
Tool	Power BI Desktop
Data Modeling	Power Query (M) for transformation, DAX for daily aggregation
Sources	Daily work-log entries consolidated per department and refreshed into the model

Key Features

- Department-wise employee productivity tables for Credit, HMPL, Accounts Payable & Insurance, and Recovery/Reconciliation desks
- Employee-wise breakdown of pre-processing vs. post-processing item counts, calls handled, and files processed
- Customer-onboarding count by employee, with a column chart ranking daily output
- Pre- vs. post-processing split shown as a donut chart for a quick daily balance check
- Date-based filtering so any single day's productivity can be reviewed department by department

Impact

- Gave department heads a same-day view of individual and team output instead of end-of-week manual summaries
- Made uneven workload distribution across employees visible for the first time
- Replaced informal, register-based daily reporting with a consistent, comparable digital record

10. Overdue Accounts Report Generator

Python + pandas automation tool that turns messy branch CSV exports into a clean, audit-ready overdue-aging report

Problem

Branch-wise overdue and NPA tracking is typically compiled by hand — pulling account data, calculating days-overdue manually, and building a summary table in Excel for each reporting cycle. This is slow, repetitive, and error-prone at scale, especially across 100+ branches with inconsistent data entry (typos in branch names, mixed date formats, missing fields).

My Role

Sole developer, end-to-end: designed the data-cleaning logic, the aging/classification rules, the branch-wise summary aggregation, and the styled Excel export — built and tested outside working hours as a personal automation project.

Architecture

Layer	Technology
Language	Python 3.14
Data Processing	pandas — loading, cleaning, aggregation, date handling
Excel Export	openpyxl — multi-sheet export with cell styling
Environment	VS Code, run via CLI (double-click compatible)
Version Control	Git & GitHub (public repository)

Key Features

- Accepts any CSV at runtime — no hardcoded filenames, validated before processing
- Cleans real-world messy data automatically: strips/normalizes inconsistent branch names, safely converts amounts containing commas or invalid text, and parses mixed date formats (DD/MM/YYYY and YYYY-MM-DD in the same file) without crashing
- Calculates Days Overdue and classifies every account into a standard aging bucket (0-30 / 31-60 / 61-90 / 90+ Days Critical), with a dedicated “Unknown” bucket for unparseable dates instead of silently dropping records
- Builds a branch-wise summary: total overdue amount and count of critical accounts per branch
- Prints a Data Quality Summary every run, reporting exactly how many rows were dropped or flagged and why — nothing is cleaned silently
- Exports a professionally styled, two-sheet Excel workbook (Detail + Branch Summary), automatically date-stamped so previous reports are never overwritten
- Wrapped in error handling so malformed input produces a readable message instead of a crash

Testing & Validation

- Verified against a 100-row dataset deliberately engineered with missing values, mixed date formats, malformed amounts, negative values, and inconsistent branch naming — to confirm the tool handles realistic dirty data, not just clean samples
- Confirmed the cleaning logic correctly collapsed inconsistent branch entries (e.g. “ Nagpur ”, “nagpur”, “Nagpur”) into a single accurate branch total rather than splitting them

Impact

- Replaces a manual, error-prone branch-reporting process with a single, repeatable script
- Produces audit-friendly output: dated filenames, a printed data-quality summary, and no silent data loss
- Demonstrates code-based automation ability alongside the Power Platform (Power Apps / Power Automate) project work above — same goal of replacing manual processes, different tool

11. Weekly Meeting Tracker

Department-wise weekly meeting compliance tracker & monthly reporting system

Problem

A multi-branch credit co-operative society's Head Office runs recurring department-wise meetings every week (Credit, HMPL Desk, Insurance, IT, Cash Verification, COB, Recon, Accounts Payable, HR, and Recovery & Legal). Whether these meetings actually happened, were skipped, or rescheduled was tracked manually, giving management no reliable way to verify meeting compliance or measure department productivity.

My Role

Sole developer, end-to-end: designed the MySQL schema, built the PHP backend, and the AJAX-driven frontend dashboard used to log and review meetings.

Architecture

Layer	Technology
Frontend	HTML5, Tailwind CSS (CDN), Vanilla JavaScript (Fetch API)
Backend	PHP 8 (procedural, mysqli with prepared statements)
Database	MySQL (InnoDB), managed via phpMyAdmin
Local Server	XAMPP (Apache + MySQL)
Reporting	PHP-generated monthly report with PDF export

Key Features

- Weekly schedule dashboard listing every department's recurring meeting — day, time, title, and location — pulled live from a meeting_templates table
- One-click meeting logging: a date picker plus “Mark Done” button saves the instance via Fetch API without a full page reload
- Recent Instances panel showing the last 7 days of recorded meetings with a colour-coded status badge (Done / Cancelled / Scheduled) and an inline status-update dropdown
- Monthly Reports page: pick any month/year, generate a compliance summary by department, and download it as a PDF for circulation to higher authorities
- Normalized two-table schema (meeting_templates + meeting_instances) that separates the recurring schedule from the actual weekly log, keeping historical reporting straightforward

Impact

- Gave management real-time, department-wise visibility into meeting compliance instead of relying on verbal confirmation
- Created a permanent, timestamped audit trail usable in productivity and performance reviews
- Cut manual report preparation — a monthly PDF summary is generated in seconds instead of being compiled by hand

A Note on Confidentiality

These case studies describe the systems I designed and built, generalized to protect my employer's identity and any operationally sensitive details (organization name, live URLs, internal data, branch counts beyond a general range). No real employee data, credentials, tenant IDs, or internal SharePoint site information appears in this document or should appear in any screenshots inserted into it.

The Overdue Accounts Report Generator (Project 10) is a personal project built and tested with synthetic/sample data — no real organizational data was used in its development or in the public GitHub repository.

If sharing this alongside GitHub: repositories still named after or linked to the organization (via URL slugs, demo video links, or license text) should be renamed or made private, since a case study here doesn't remove information already public elsewhere.